

Ornstein-Uhlenbeck Process

หัวใจของ Stat Arb — โมเดลที่อธิบายว่า spread "วนกลับ" อย่างไร
 κ (speed) · θ (mean) · σ (noise) · Half-life · Calibration

แก่นของบทนี้ — อ่านก่อน

OU Process คือ "สปริงที่มี noise" — ยิ่งดึง spread ออกจากค่ากลาง สปริงยิ่งดึงกลับแรง

Half-life บอกว่า spread ใช้เวลากลับครึ่งทางนานแค่ไหน → กำหนดว่าเราจะ trade แบบ intraday หรือ swing

$$d\varepsilon = \kappa(\theta - \varepsilon) dt + \sigma dw$$

3.1 Parameters ของ OU Process

Parameter	ชื่อ	ความหมาย trade	ค่าปกติ
$\kappa > 0$	Mean-reversion speed	ยิ่งมาก spread กลับเร็ว → ต้อง execute เร็ว	1–5 ต่อวัน
θ	Long-run mean	ค่ากลางที่ entry/exit อิงอยู่ — ถ้าเปลี่ยนบ่อยต้อง re-calibrate	≈ 0 หรือ avg funding diff
$\sigma > 0$	Diffusion (noise)	ยิ่งมาก spread ผันผวนมาก → threshold ต้องกว้างกว่า fee	0.01–0.5% ต่อ $\sqrt{\text{วัน}}$

3.2 OU Solution — ϵ เดินไปไหนได้บ้าง

SDE นี้ solve ได้ closed-form ทำให้เราคำนวณ expected value ได้ทุกเวลา:

$\epsilon_t = \theta + (\epsilon_0 - \theta) \cdot e^{-kt} + \sigma \cdot \int_0^t e^{-k(t-s)} dw_s$ ↑ ↑ ↑ ค่ากลาง decay จากจุดเริ่ม noise สะสม

$$E[\epsilon_t | \epsilon_0] = \theta + (\epsilon_0 - \theta) \cdot e^{-kt}$$

เริ่มที่ $\epsilon_0 = \theta + \text{gap}_0$ (ห่างจากค่ากลาง gap_0)

หลังเวลา t : ϵ คาดว่าจะอยู่ที่ $\theta + \text{gap}_0 \cdot e^{-kt}$

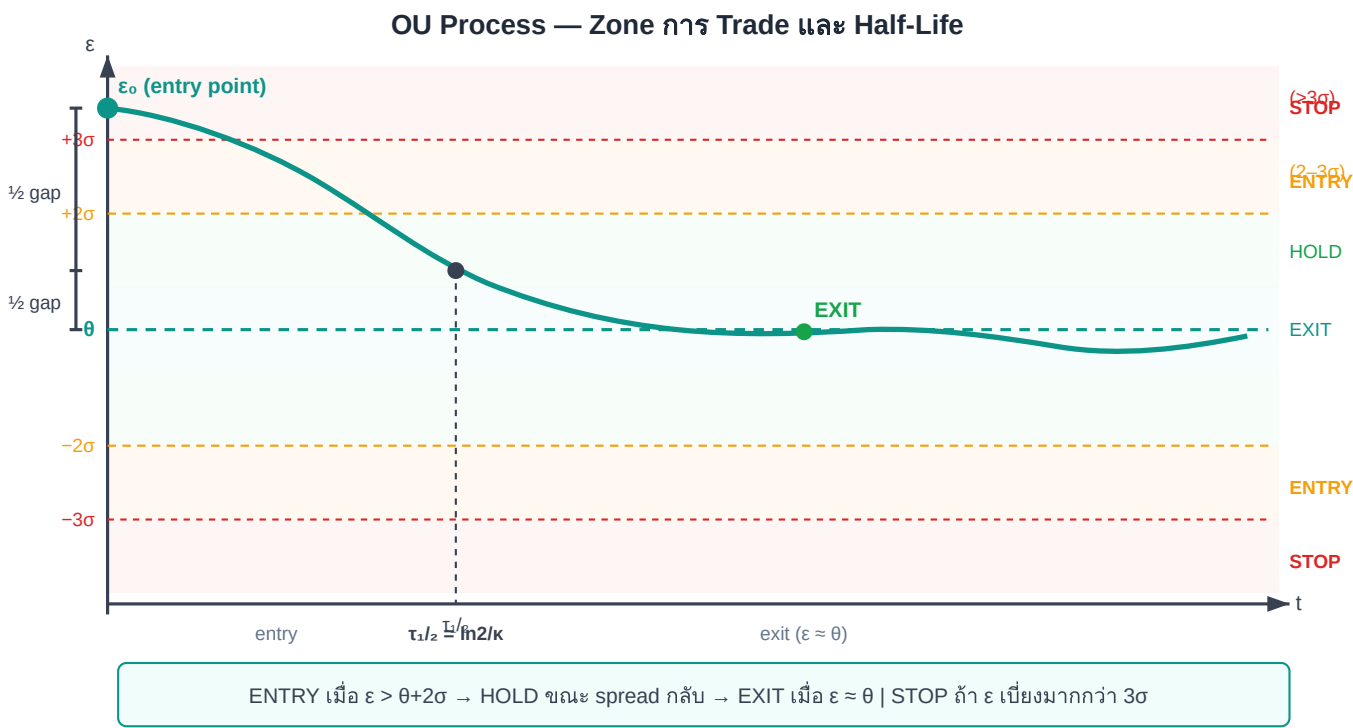
gap ลดลงด้วยอัตรา e^{-kt} → ยิ่งนาน ยิ่งใกล้ θ

3.3 Half-Life — ตัวเลขที่สำคัญที่สุดสำหรับ Trader

HALF-LIFE

$\tau_{1/2} = \ln(2) / \kappa \approx 0.693 / \kappa$

เวลาที่ gap ระหว่าง ϵ และ θ ลดลงครึ่งหนึ่ง
ถ้า spread เพียง 1% จาก $\theta \rightarrow$ หลัง $\tau_{1/2}$ จะเหลือ 0.5%



Half-life = ช่วงเวลาที่ spread เดินทางครึ่งทางกลับ — ใช้วางแผน expected holding time

Half-life กับประเภท Strategy

κ (ต่อวัน)	Half-life	ประเภท Trade	ข้อกำหนด Execution
0.1	~7 วัน	Position trade / swing	ไม่เร่ง ต้องมี margin buffer
0.5	~33 ชม.	Swing ข้ามวัน	Monitor วันละครั้ง
1–2	8–17 ชม.	Intraday (เหมาะสมสุด)	Check ทุก 1–2 ชม.
5	~3 ชม.	Fast intraday	Algo-assisted execution

10+	<2 ชม.	Near HFT	ต้องการ co-location, API
-----	--------	----------	--------------------------

3.4 Stationary Distribution — Band ของ Trade

เมื่อ $t \rightarrow \infty$ OU process เข้าสู่ stationary distribution ซึ่งเป็น Normal:

$$\varepsilon_\infty \sim N(\theta, \sigma^2/(2\kappa)) \quad \sigma_\varepsilon = \sigma/\sqrt{2\kappa} \leftarrow \text{stationary std dev (ใช้ตั้ง entry band)}$$

σ_ε ใช้ทำอะไร?

- Entry threshold: $\varepsilon > \theta + 2\sigma_\varepsilon$ หรือ $\varepsilon < \theta - 2\sigma_\varepsilon$ (เกิดราว 5% ของเวลา)
- Exit threshold: $\varepsilon \approx \theta \pm 0.5\sigma_\varepsilon$
- Stop-loss: $|\varepsilon - \theta| > 3\sigma_\varepsilon$ (spread "หลุด" ผิดปกติ)
- ตรวจสอบ: ε ควรอยู่ในช่วง $\theta \pm 3\sigma_\varepsilon$ กว่า 99.7% ของเวลา — ถ้าไม่ใช่ = signal ให้ re-calibrate

3.5 AR(1): OU Process ในโลกข้อมูล Discrete

ข้อมูลจริงเป็น discrete (รายนาที รายชั่วโมง) เมื่อแปลง OU เป็น AR(1):

OU → AR(1) (TIME STEP Δt)

$$\varepsilon_t = a + b \cdot \varepsilon_{t-1} + \eta_t$$

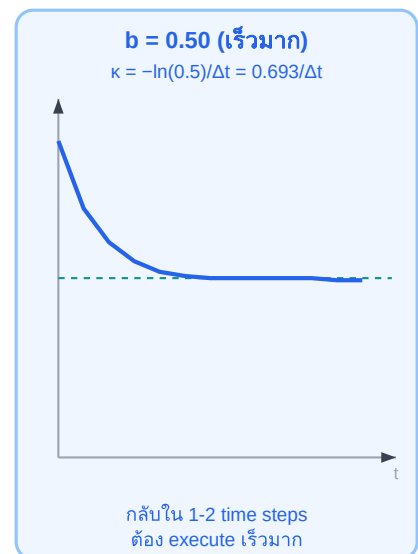
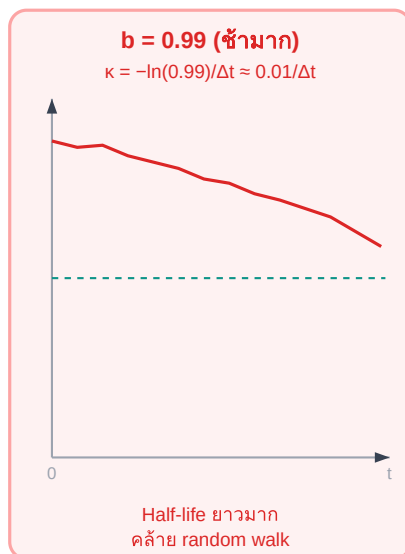
$b = e^{-\kappa \Delta t}$ ← mean-reversion coefficient

$a = \theta(1-b)$ ← intercept

$\eta_t \sim N(0, \sigma^2(1-b^2)/(2\kappa))$ ← noise

อ่าน b บน Scale — b บอกอะไร?

ค่า b ใน AR(1) บอกความเร็ว mean-reversion



$b \rightarrow 1.0$ (ช้า, random walk)

$b \approx 0.9$ (sweet spot)

$b \rightarrow 0$ (เร็วมาก, white noise)

$b \approx 0.9-0.95$ ต่อ time step มักเป็น sweet spot สำหรับ intraday stat arb

สูตรแปลง AR(1) → OU PARAMETERS

$$\kappa = -\ln(b) / \Delta t \quad \theta = a / (1 - b)$$

ตัวอย่าง: $b = 0.90$, $\Delta t = 1$ ชม. = $1/24$ วัน

$\kappa = -\ln(0.90) / (1/24) \rightarrow -\ln(0.90) = \mathbf{0.1054} \rightarrow 0.1054 \times 24 = \mathbf{2.53}$ ต่อวัน

Half-life = $\ln(2) / 2.53 = \mathbf{0.274}$ วัน = $\mathbf{6.6}$ ชม.

3.6 Calibration — หา κ , θ , σ จากข้อมูลจริง

เราเพิ่งเรียนรู้ว่า AR(1) ที่ fit บน spread ε_t ก็คือ OU process ในรูปแบบ discrete — ดังนั้นวิธีหา κ , θ , σ จากข้อมูลจริงคือ **fit AR(1)** แล้วแปลงค่า ตามขั้นตอนนี้:

1 สร้าง spread series: $\varepsilon_t = \log(P_{\text{Bybit}_t}) - \beta \cdot \log(P_{\text{Lighter}_t})$
(β ได้จาก OLS บน log returns — [บทที่ 4](#))

2 Regress AR(1): $\varepsilon_t = a + b \cdot \varepsilon_{t-1} + \eta_t$
บันทึก: a , b , และ residuals η

3 คำนวณ OU parameters:
 $\theta = a/(1-b)$, $\kappa = -\ln(b)/\Delta t$, $\sigma_{\text{stat}} \approx \text{std}(\varepsilon - \theta)$
หมายเหตุ: $\text{std}(\varepsilon - \theta)$ เป็นตัวประมาณค่า (sample estimator) ของ $\sigma_{\text{stat}} = \sigma/\sqrt{(2\kappa)}$ ตัวเดียวกันจาก §3.4 — ทั้งสองค่าจะลู่เข้าหากันเมื่อจำนวนข้อมูลมากพอ

4 คำนวณ trade parameters:
 $\tau_{1/2} = \ln(2)/\kappa \rightarrow$ กำหนด holding time
Entry band = $\theta \pm 2\sigma_{\text{stat}} \rightarrow$ กำหนด entry threshold

```
# Pseudo-code: OU Calibration
function calibrate_ou(eps_series, delta_t):
    # Step 1: AR(1) regression
    y = eps_series[1:] #  $\varepsilon_t$ 
    x = eps_series[:-1] #  $\varepsilon_{t-1}$ 
    b, a = ols_slope_intercept(x, y)
    residuals = y - (a + b * x)

    # Step 2: OU parameters
    theta = a / (1 - b)
    kappa = -log(b) / delta_t
    sigma_stat = std(eps_series - theta)
    half_life = log(2) / kappa

    # Step 3: Trade bands
    entry_upper = theta + 2 * sigma_stat
    entry_lower = theta - 2 * sigma_stat
    stop_upper = theta + 3 * sigma_stat
```

```
stop_lower = theta - 3 * sigma_stat

return {theta, kappa, sigma_stat, half_life,
        entry_upper, entry_lower, stop_upper, stop_lower}
```

ตัวอย่างจริง: Bybit ↔ Lighter BTC — Calibration ผล

ข้อมูล: 30 วัน, รายนาที่ ($\Delta t = 1/1440$ วัน) $\varepsilon_t = \log(P_{\text{Bybit}}) - 1.003 \cdot \log(P_{\text{Lighter}})$ OLS AR(1) result: $b = 0.9985$, $a = 2.0 \times 10^{-6}$ Residual std = 0.00012 OU Parameters: $\theta = 2.0e-6 / (1-0.9985) = 0.00133$ (basis $\approx 0.133\%$) $\kappa = -\ln(0.9985)/(1/1440) = 2.16$ ต่อวัน $\sigma_{\text{stat}} \approx 0.00080$ ($\approx 0.08\%$) Trade Parameters: $\tau_{1/2} = 0.693/2.16 = 7.7$ ชั่วโมง ← intraday trade ได้ Entry: $\varepsilon > 0.293\%$ หรือ $\varepsilon < -0.027\%$ ($\theta \pm 2 \times 0.08\%$) Stop: $\varepsilon > 0.373\%$ หรือ $\varepsilon < -0.107\%$ ($\theta \pm 3 \times 0.08\%$)

สรุป: spread กลับใน ~8 ชม. เปิด position ตอนเช้า → ปิดก่อนค่ำ

3.7 เมื่อไร OU เดียวไม่พอ — และบทถัดไปจะแก้อย่างไร

สมมติฐาน OU	เมื่อไรที่ผิด	สัญญาณที่เห็น	โมเดลที่ใช้แทน
κ, θ คงที่	ตลาด regime change: funding rate เปลี่ยนโครงสร้าง	θ drift ออก, ε ไม่กลับ	Kalman (Ch.15), Regime-Switch (Ch.9)
σ คงที่	ช่วง volatile: market crash, news	residual variance ฟุ้ง	GARCH on residuals (Ch.7)
Gaussian noise	liquidation cascade, news spike	residual มี fat tail, jump	Jump-Diffusion OU (Ch.8)
β คงที่	ตลาดเปลี่ยน correlation	ε drift กลับมา	Kalman Filter β (Ch.15)

Rule of thumb: เมื่อไรต้อง re-calibrate?

- θ เคลื่อนออกจากค่าเดิม $> 1\sigma_{\text{stat}}$ ใน 5 วัน $\rightarrow$ re-calibrate β และ θ
- Residual variance ฟุ้ง $> 2\times$ baseline $\rightarrow$ switch ไป GARCH
- Half-life ยาวขึ้น $> 2\times$ $\rightarrow$ regime เปลี่ยน พิจารณา exit ทั้งหมด

3.8 ตารางสัญลักษณ์ (Notation Reference) — ใช้ตลอดทั้งเล่ม

ทั้งเล่มใช้ σ หลายแบบ ต่างกันที่จุดประสงค์และวิธีคำนวณ — ตารางนี้เป็นอ้างอิงกลางที่ควรกลับมาดูทุกครั้งที่สับสน:

สัญลักษณ์	ชื่อเต็ม	ความหมาย	คำนวณจาก	ใช้ใน
σ (SDE)	Diffusion coefficient	ขนาด noise ใน OU SDE: $d\varepsilon = \kappa(\theta - \varepsilon)dt + \sigma \cdot dW$	$\sigma = \sigma_{\text{stat}} \cdot \sqrt{(2\kappa)}$	Ch.2–3 สมการทฤษฎี
σ_{stat}	Stationary std	Std ของ spread ε ณ steady state — ใช้กำหนด entry/exit band	$\sigma_{\text{stat}} = \sigma / \sqrt{(2\kappa)} \approx \text{std}(\varepsilon - \theta)$	Ch.3–4 entry band $\theta \pm 2\sigma_{\text{stat}}$
σ_t (GARCH)	Conditional std	Std ของ spread ε ณ เวลา t — ขยายตัวช่วง volatile	$\sigma_t^2 = \omega + \alpha \cdot \varepsilon_{t-1}^2 + \beta_G \cdot \sigma_{t-1}^2$	Ch.7 GARCH z-score
σ_{robust}	Robust std (MAD-based)	Estimate σ_{stat} ที่ทนต่อ outlier — ไม่กระทบจาก jump	$\sigma_{\text{robust}} = \text{MAD} \times 1.4826$	Ch.10 robust z-score
σ_{EMA}	EMA-smoothed std	Rolling std ด้วย exponential weighting — Welford-style	Welford online update (Ch.15)	Ch.15 dynamic β tracking

กฎง่าย ๆ ในการเลือก σ

- ตลาดปกติ: ใช้ σ_{stat} (OLS residual std) — เร็วและง่าย
- ช่วง volatile (ratio 7d/30d std > 1.5): เปลี่ยนเป็น σ_t จาก GARCH(1,1)
- มี outlier / jump: ใช้ $\sigma_{\text{robust}} = \text{MAD} \times 1.4826$ แทน std()
- β เปลี่ยนตาม Kalman: ε เปลี่ยนทุก bar $\rightarrow$ ใช้ σ_{EMA} แบบ Welford update

สัญลักษณ์อื่น ๆ	ความหมาย
ε_t	Spread residual at time t : $\varepsilon = \log(P_A) - \beta \cdot \log(P_B)$
θ	Long-run mean ของ OU (equilibrium spread level)
κ	Mean-reversion speed (ต่อวัน); $\tau_{1/2} = \ln(2)/\kappa$
β	Hedge ratio (OLS, TLS, หรือ Kalman)

H	Hurst exponent; $H < 0.5$ = mean-reverting
z_t	Z-score ของ spread: $(\varepsilon_t - \theta)/\sigma_{\text{stat}}$ หรือ $(\varepsilon_t - \theta)/\sigma_t$
$\tau_{1/2}$	Half-life ของ mean-reversion: $\tau_{1/2} = \ln(2)/\kappa$
Δt	Bar size ในหน่วยวัน (1-hour bar: $\Delta t = 1/24$)

3.9 Walk-Forward Calibration — ป้องกัน Overfitting

แก่นความคิด — อ่านก่อน

การ backtest แบบ **Static Window** คือดู historical data ครั้งเดียว → รู้อนาคตโดยไม่รู้ตัว (lookahead bias)

Walk-Forward แก้ปัญหานี้โดยแบ่งเวลาแบบ rolling ลำดับเสมอ — เราใช้ข้อมูลอดีตเพื่อ fit แล้วทดสอบบนข้อมูลอนาคตที่โมเดลยังไม่เคยเห็น

Concept Drift — ทำไม Parameters เปลี่ยนตลอดเวลา

OU parameters (k , θ , σ) ไม่คงที่ตลอดเวลา ตลาดเปลี่ยนแปลงตามสถานะเศรษฐกิจ ความเชื่อมั่นของนักลงทุน และโครงสร้าง funding rate — สิ่งนี้เรียกว่า **Concept Drift**

- **Static window:** fit OU บนข้อมูลทั้งหมดครั้งเดียว → parameters เป็นค่าเฉลี่ยของทุกช่วงเวลา → overfits กับช่วงอดีตที่ตลาดอาจไม่มีอีกแล้ว
- **Walk-Forward:** fit OU บน window อดีต → apply บน window อนาคตที่ยังไม่เคยเห็น → สะท้อน parameters ของตลาด ณ ช่วงนั้นจริง ๆ

อันตรายของ Static Calibration

- θ ที่ fit ได้ อาจเป็นค่าเฉลี่ยของ regime หลายช่วงที่ไม่เคยเกิดพร้อมกัน
- k ที่ได้ อาจสูงเกินจริง เพราะ data ในช่วง volatile ดูเหมือน mean-revert เร็ว
- เมื่อ trade จริง parameters เหล่านั้นไม่ตรงกับตลาดปัจจุบัน → signal ผิด → ขาดทุน

Walk-Forward Algorithm — 3 ขั้นตอนหลัก

1 แบ่ง Window: Train window = 90 วัน, Test window = 30 วัน
เริ่มที่วันที่ 1–90 เป็น train, วันที่ 91–120 เป็น test — ห้าม shuffle หรือ random split เด็ดขาด ต้องเรียงตามเวลาเสมอ

2 Fit แล้ว Apply: Calibrate OU parameters บน train window → นำ parameters ไป generate signals บน test window → บันทึก PnL ของ test window
ไม่มี lookahead: test window ถูก "เปิดเผย" ทีละ bar ไม่ใช่ทั้งหมด

3

เลื่อน Window: ขยับ window ไป 30 วัน (= test_days) → วงซ้ำขั้นตอน 1–2

แต่ละรอบให้ PnL 1 window — รวมทุก window ได้ equity curve ที่ไม่มี lookahead

Walk-Forward — Rolling Windows ตามเวลา



แต่ละ train block (สีน้ำเงิน) ให้ parameters → apply บน test block (สีเขียว) ที่อยู่ถัดไปในอนาคต — เลื่อนซ้ำตลอดช่วงข้อมูล

Pseudo-code — Walk-Forward Calibration

```
def walk_forward_calibration(prices_A, prices_B, train_days=90, test_days=30):
    results = []
    i = 0
    while i + train_days + test_days <= len(prices_A):
        train_A = prices_A[i : i+train_days]
        train_B = prices_B[i : i+train_days]
        test_A = prices_A[i+train_days : i+train_days+test_days]
        test_B = prices_B[i+train_days : i+train_days+test_days]
        # Fit OU parameters on train
        beta = estimate_beta_ols(train_A, train_B)
        epsilon_train = log(train_A) - beta * log(train_B)
        kappa, theta, sigma_stat = calibrate_ou(epsilon_train)
        # Apply on test (no lookahead)
        epsilon_test = log(test_A) - beta * log(test_B)
        signals = generate_signals(epsilon_test, kappa, theta, sigma_stat)
        pnl = simulate_pnl(signals, test_A, test_B)
        results.append({"window_start": i, "beta": beta, "kappa": kappa, "pnl": pnl})
        i += test_days # slide forward
    return results
```

จุดสำคัญที่ต้องระวัง

- $i += \text{test_days}$ — เลื่อน window ด้วย test_days ไม่ใช่ 1 วัน เพื่อให้แต่ละ test window ไม่ overlap กัน
- Train window ขนาด 90 วัน ต้องมี data เพียงพอสำหรับ AR(1) regression — น้อยกว่า 30 obs จะไม่ reliable
- ห้าม reuse test data เป็น train ของ window เดิม — window ต้องเลื่อนเป็นลำดับเสมอ

ตัวอย่างจริง: Bybit ↔ Lighter BTC — Walk-Forward 360 วัน

ใช้ข้อมูล 360 วัน → walk-forward ได้ **9 windows** (train=90d, test=30d, slide=30d)

Window 1: train day 1-90, test day 91-120 → half-life = 6.1h Window 2: train day 31-120, test day 121-150 → half-life = 5.2h Window 3: train day 61-150, test day 151-180 → half-life = 8.7h Window 4: train day 91-180, test day 181-210 → half-life = 9.4h Window 5: train day 121-210, test day 211-240 → half-life = 7.1h Window 6: train day 151-240, test day 241-270 → half-life = 11.3h Window 7: train day 181-270, test day 271-300 → half-life = 6.8h Window 8: train day 211-300, test day 301-330 → half-life = 5.9h Window 9: train day 241-330, test day 331-360 → half-life = 6.5h เฉลี่ย: 7.4h | Range: 5.2-11.3h

Half-life เฉลี่ยจาก 9 windows = **7.4 ชั่วโมง** (range: 5.2-11.3h) — ความผันผวนของ half-life ระหว่าง windows แสดงว่า OU parameters เปลี่ยนตามช่วงเวลาจริง → **justifies การใช้ walk-forward แทน static calibration** ถ้าใช้ static จะได้ค่าเดียวตลอด ซึ่งผิดพลาดในหลายช่วง

โจทย์ 3.4 — Walk-Forward Window Count

มีข้อมูล **180 วัน**, ตั้งค่า train = 90 วัน, test = 30 วัน, slide = 30 วัน

ถาม: (a) ได้กี่ windows? (b) first trade เริ่มวันที่เท่าไร? (c) ถาลด train เหลือ 60 วัน จะได้กี่ windows?

► คำตอบ

โจทย์ 3.1 — คำนวณ half-life และ trade parameters

OLS AR(1) บน spread รายชั่วโมง ($\Delta t = 1/24$ วัน): $b = 0.94$, $a = 0.00012$
 $\text{std}(\text{residuals}) = 0.00015$

ถาม: (a) k ต่อวัน (b) half-life เป็นชั่วโมง (c) entry band (d) spread ปัจจุบัน 0.05% จาก θ — ควร enter ไหม?

► คำตอบ

โจทย์ 3.2 — แปลง b → strategy design

Spread รายชั่วโมง ให้ผล AR(1): $b = 0.97$

ค่า fee รวม (open+close) = 0.06% per round-trip

ถาม: (a) half-life (b) ถ้าตั้ง entry ที่ $2\sigma_{\text{stat}} = 0.10\%$ จะถือ position นานแค่ไหนโดยเฉลี่ย และ expected profit คืออะไร (ก่อน fee)?

► คำตอบ

โจทย์ 3.3 — วินิจฉัย OU breakdown

ใช้ OU trade Bybit $\leftrightarrow$ Lighter มา 2 สัปดาห์ k เดิม = 2 ต่อวัน, $\theta = 0.1\%$

สัปดาห์ที่ 3: spread ออกไปถึง 0.4% และค้างอยู่นาน 3 วัน ไม่กลับ

ถาม: เกิดอะไรขึ้น และควรทำอะไร?

► คำตอบ